HeimWatt

Documentation

Technical & Operational Reference

Version 1.0.0+1 | April 2, 2026

Document Purpose	Technical and operational reference for accepting ownership of the HeimWatt Flutter application.
Application	Customer-facing portal for solar/installation project documentation: photo capture, guided installation steps, address selection, PDF generation, and integration with Heim-Watt backend APIs and HubSpot (via Cloud Functions proxy).
Framework	Flutter: cross-platform (primary Web, with Android/iOS project folders present)
Version	1.0.0+1 (from pubspec.yaml)
Last Updated	April 2, 2026

Table of Contents

1 Executive Summary

HeimWatt is a cross-platform Flutter application (primary deployment: Web, with Android/iOS project folders present) that allows authenticated users to:

- Sign in via Firebase Authentication (email/password or custom token flow for magic-link style deep links).
- Associate their session with a HubSpot deal and contact (loaded through the Heim-Watt API).
- Complete a multi-step installation workflow: project overview, tutorials, address selection (Google Maps), media library/bulk upload, and per-step photo documentation with optional image editing and video instructions.
- Generate PDF reports from installation data and upload documents to HubSpot-related endpoints.

> *Persistent state is stored in Cloud Firestore (users and projects) and Firebase Storage (images). API calls use Bearer tokens — the Firebase ID token stored as the app's access token for REST calls.*

2 Technology Stack

Framework	Flutter (Dart SDK ^3.10.0-290.4.beta per pubspec.yaml)
State / DI	GetX (get)
Routing	go_router with GetMaterialApp.router
Backend Services	Firebase Core, Auth, Firestore, Storage
HTTP Client	Dio (ApiClient in lib/app/services/api_services.dart)
Local Preferences	shared_preferences
Sensitive Tokens	flutter_secure_storage (access/refresh token keys)
UI Helpers	responsive_builder (mobile/tablet/desktop layouts), toastification, gap, Manrope font family
Maps	google_maps_flutter (web loads Maps JS API in web/index.html)
PDF	pdf, printing, syncfusion_flutter_pdf / syncfusion_flutter_pdfviewer
Media	image_picker, file_picker, video_player, ffmpeg_kit_flutter_new, pro_image_editor, flutter_image_compress
i18n	flutter_intl enabled in pubspec.yaml; generated code under lib/generated/

3 High-Level Architecture

```
Flutter app (Heimwatt)
  GoRouter (auth gates)  ->  Screens  ->  GetX Controllers
  Responsive layouts: mobile / tablet / desktop per module

          |                                |
          v                                v

Firebase Auth                   Heim-Watt REST API (staging)
Firestore (users,               + HubSpot proxy (Cloud Functions)
 project, install_form)           for files, notes, deal patch
Firebase Storage (images)
```

- **Routing and auth:** AppRoutes (lib/app/routes/app_routes.dart) uses GoRouter with a redirect that requires both FirebaseAuth.instance.currentUser and a non-empty PrefService.userId for protected routes. Public routes: auth (login entry), forgot password, OTP, reset password.
- **Repository:** MainRepository (lib/repository/main_repository.dart) wraps ApiClient for deals, contacts, PDF upload, notes, and document linking.
- **Installation workflow:** Central logic lives in InstallationStepsController (lib/app/modules/installation_steps/installation_steps_controller.dart) — a large controller coordinating Firestore reads/writes, Storage uploads, PDF generation (PdfGenerationService), and HubSpot-related uploads.

4 Repository Directory Structure

Only the lib/ tree is summarized here; platform folders (android/, ios/, web/, etc.) follow the standard Flutter layout.

lib/main.dart	App entry: PrefService.init(), Firebase init, Heimwatt root widget, ToastificationWrapper, GlobalZoomService
lib/firebase_options.dart	FlutterFire-generated platform options (web, Android, iOS); macOS/Windows throw UnsupportedError if used without extending
lib/app/routes/app_routes.dart	Route constants, GoRouter definition, auth redirect
lib/repository/main_repository.dart	API methods: deals, contacts, location patch, file upload, notes, documents
lib/app/services/	api_services.dart (Dio), pdf_generation_service.dart
lib/app/utils/	pref_service.dart, secure_token_service.dart, magic_link_service.dart, app_functions.dart, api_endpoints.dart (minimal), exports.dart
lib/app/theme/	colors.dart, text_styles.dart, app_strings.dart, api_endpoints.dart (actual REST base URLs — see Section 6)

lib/app/exception/	app_exception.dart
lib/app/modules/auth/	Login, authentication (token route), forgot password, OTP, reset password
lib/app/modules/deal_selection/	Deal list/selection when user has no dealId in preferences
lib/app/modules/installation_steps/	Main feature: screen, controller, responsive views, subflows (project, tutorial, address, step type, media library, installation form, PDF previews)
lib/app/data/common_widget/	Shared buttons, dialogs, scrollables
lib/gen/assets.gen.dart	Generated asset references (flutter_gen)

5 Application Flows (Functional)

5.1 Authentication

- **Email / password:** LoginController.login calls Firebase signInWithEmailAndPassword, creates a user doc in Firestore if new, then persists userId and accessToken (Firebase ID token) via PrefService. Then either loads deal if dealId is set, or navigates to deal selection.
- **Magic link / custom token:** Route /login/:token calls AuthenticationScreen, which uses MagicLinkService.loginWithToken with signInWithCustomToken, then follows the same persistence and getDealById/getContactById flow as applicable.

5.2 Post-Login

- InstallationStepsScreen registers InstallationStepsController and picks mobile/tablet/desktop views via ScreenTypeLayout.
- Sub-flow flags on the controller include: project screen, tutorial, address selection, step type selection, media library, installation form (see controller fields such as showProjectScreen, showAddressSelectionScreen, etc.).

5.3 Data Model (Firestore)

users	Per Firebase UID: email, role, approval, display name, HubSpot contact reference fields, etc.
project	One project per user (queried by user_id): address, installation_steps, product-specific maps (heatpump, combined, photovoltaics), bulk_import/media library, pdf_url, metadata
install_form	Template for installation step structure; controller syncs project installation_steps from this template (see installation_steps_controller.dart and login_controller.dart initialization)

Firebase Storage: Used for user/project image uploads (paths resolved in InstallationStepsController).

5.4 External APIs

- Heim-Watt API (currently staging host in code) for deals, contacts, deal documents.
- Google Cloud Function HubSpot proxy for HubSpot CRM/files endpoints (upload file, create note, patch deal).

6 Configuration and Endpoints

6.1 API Base URLs (lib/app/theme/api_endpoints.dart)

- **Heim-Watt API (staging):** https://api.staging.heim-watt.de/v1/... — deals, contacts, documents.
- **HubSpot proxy:** https://europe-west3-heimwatt-app-ffe6c.cloudfunctions.net/hubspotProxy/... — files, notes, deal patch for linking files.

Older commented URLs (Azure adapter, alternate deal URL) are preserved in-file for reference. Production cutover should be a coordinated change here, plus backend/CORS and Firebase rules review.

6.2 Firebase (firebase.json)

Project ID	heimwatt-staging
Hosting public	build/web
Site name	project-photo-upload-staging
Routing	SPA rewrite to index.html

6.3 Web (web/index.html)

Loads Firebase JS (compat) and Google Maps with a browser API key in the script URL.

> *Security note: Maps keys in client HTML are visible to end users. Restrict the key in Google Cloud Console (HTTP referrers, APIs used) and avoid committing unrestricted production keys to public repos.*

6.4 Tokens

- PrefService.accessToken is backed by secure storage; it is used as Authorization: Bearer for ApiClient requests.
- After login, this is typically the Firebase ID token (see login_controller.dart / magic_link_service.dart).

7 Key Dependencies

App shell	get, go_router, responsive_builder, toastification
Network / data	dio, shared_preferences, flutter_secure_storage, cached_network_image
Firebase	firebase_core, firebase_auth, cloud_firestore, firebase_storage
PDF / document	pdf, printing, syncfusion_flutter_pdf, syncfusion_flutter_pdfviewer
Maps / location	google_maps_flutter
Media	image_picker, file_picker, video_player, ffmpeg_kit_flutter_new, pro_image_editor, screenshot, flutter_image_compress

Run flutter pub get after cloning. Use build_runner / flutter_gen only if regenerating assets (flutter_gen_runner is in dev_dependencies).

8 Build, Run, and Deploy

8.1 Prerequisites

- Flutter SDK compatible with the Dart SDK constraint in pubspec.yaml
- Firebase CLI (for hosting deploy)
- Platform toolchains if building Android/iOS (not fully documented in-repo)

8.2 Local Run

```
flutter pub get
flutter run -d chrome
```

For Android/iOS, use a configured device or emulator and ensure google-services.json / GoogleService-Info.plist match the Firebase project.

8.3 Web Production Build

```
flutter build web --release
```

Output: build/web/. Deploy to Firebase Hosting per firebase.json (or another static host with SPA routing).

firebase projects:list

flutter build web --release

firebase deploy

firebase use <project id>

Special instruction for deploy :

1. in firebase. Json file I set site and mention client's firebase project id so it will every time deploy on this particular domain.

2. after run this command firebase deploy --only hosting: project-photo-upload-staging.

3. URL : https://erfassung.staging.heim-watt.de/

8.4 Analyze

```
flutter analyze
```

analysis_options.yaml extends package:flutter_lints/flutter.yaml.

9 Security and Compliance Checklist

1. **Secrets:** Rotate any keys that were exposed in repositories; restrict Google Maps and Firebase console settings (domains, bundle IDs, SHA fingerprints).
2. **API environment:** Confirm whether production should use api.staging.heim-watt.de or a production host; update api_endpoints.dart and test all flows (deal load, document upload, HubSpot proxy).
3. **Firestore / Storage rules:** Review in Firebase Console for least privilege (authenticated users, project ownership).
4. **Logging:** ApiClient logs URLs and tokens in development-style log() calls — review for production builds (reduce verbosity, avoid logging full tokens).
5. **CORS / cookies:** Web app relies on Firebase and REST calls; ensure backend allows your deployed origin.

10 Testing and Quality

flutter_test is listed in pubspec.yaml. Integration/widget tests should be added as part of ongoing maintenance if coverage is required for release gates.

Manual Regression Paths

- Login (email + magic link)
- Deal selection
- Full installation step upload
- PDF generate/upload
- Logout/session

11 Known Implementation Notes

- **firebase_options.dart:** Default options throw on macOS and Windows in the generated file — desktop Firebase use would require FlutterFire reconfiguration.
- **lib/app/utils/api_endpoints.dart** is largely empty; **lib/app/theme/api_endpoints.dart** holds the real ApiEndpoints class used by MainRepository.
- **Navigation:** InstallationStepsScreen uses PopScope(canPop: false) to discourage back navigation while authenticated.
- **Commented code:** magic_link_service.dart contains commented URL parsing for deep-link testing; production behavior is via /login/:token.

12 Contact and Ownership Handover

For ongoing operations, the client should obtain:

- Access to the Firebase project (heimwatt-staging or successor) and Hosting site.
- Heim-Watt API credentials, environment URLs, and HubSpot integration contracts.
- Google Cloud project for Maps and any Cloud Functions (hubspotProxy region: europe-west3).
- Source control (this repository) and CI/CD definitions if used outside this tree.